RTU B.TECH V SEMESTER

AID / CAI / CDS

DATA MINING

Concepts and Techniques (5AID3-01)

UNIT 2

Predictive Modeling

Classification & Prediction Techniques

★ PYQ-Focused Complete Study Notes ★

Decision Trees • Bayes Classification • Bayesian Belief Networks
Backpropagation • Support Vector Machines • Lazy Learners

Credit	Max Marks	Exam Pattern	Duration
2 (2L+0T+0P)	100 (IA:30, ETE:70)	A:10×2 | B:5×4 | C:3×10	3 Hours

TABLE OF CONTENTS

Legend & Symbol Guide **3**

PART 1 — Topic-Wise Notes **4**

1. General Approach to Classification 4
2. Decision Tree Induction 6
3. Bayes Classification Methods (Naïve Bayes) 8
4. **Bayesian Belief Networks ⍰ Very High** **11**
5. Classification by Backpropagation 13
6. Support Vector Machines (SVM) 15
7. Lazy Learners (vs Eager Learners) 17

PART 2 — Complete Unit 2 PYQ Bank **19**

PART 3 — Model Answers **21**

PART 4 — Final Revision Material **23**

RTU DATA MINING (5AID3-01)

Unit 2 — Complete Study Notes

Predictive Modeling — Classification & Prediction Techniques

PYQ-Focused | B.Tech V-Sem AID / CAI / CDS

Credit	Max Marks	Exam Pattern	Duration
2 (2L+0T+0P)	100 (IA:30, ETE:70)	Part A:10×2 | Part B:5×4 | Part C:3×10	3 Hours

How to use this document Headings are Word-navigable — open the Navigation Pane (View → Navigation Pane) to jump between topics instantly. Each topic ends with a Quick Revision Box — read only these the night before the exam. tags beside topic titles show PYQ frequency — study Very High topics first. This unit is numerical-heavy — practice every solved example by hand at least once.

Legend & Symbol Guide

Symbol	Meaning
Very High	Asked repeatedly across papers — must-know
High	Asked in 2 papers — high scoring chance
Medium	Asked once — good to know well
○Low	Not asked yet — brief coverage only
⋆Important Concept	Core definition — memorize word-for-word
RTU Exam Point	Exam-specific tip or expected marks split
Frequently Asked	PYQ frequency tag beside heading
Common Mistake	Where students typically lose marks
Numerical	Solved example — formula-based/algorithmic
Quick Revision	End-of-topic summary box

PART 1 — TOPIC-WISE NOTES

1. General Approach to Classification (incl. Issues in Classification & Prediction)

PYQ Frequency: Medium | RTU 2025 — foundational, extendable to 4m/10m

PYQs from This Topic

2025 — 2 Marks: What is classification in predictive modeling?

Expected (teacher bank, matches older RTU pattern) — 10 Marks: Write and explain the concept of classification and prediction, along with various issues incorporated with them.

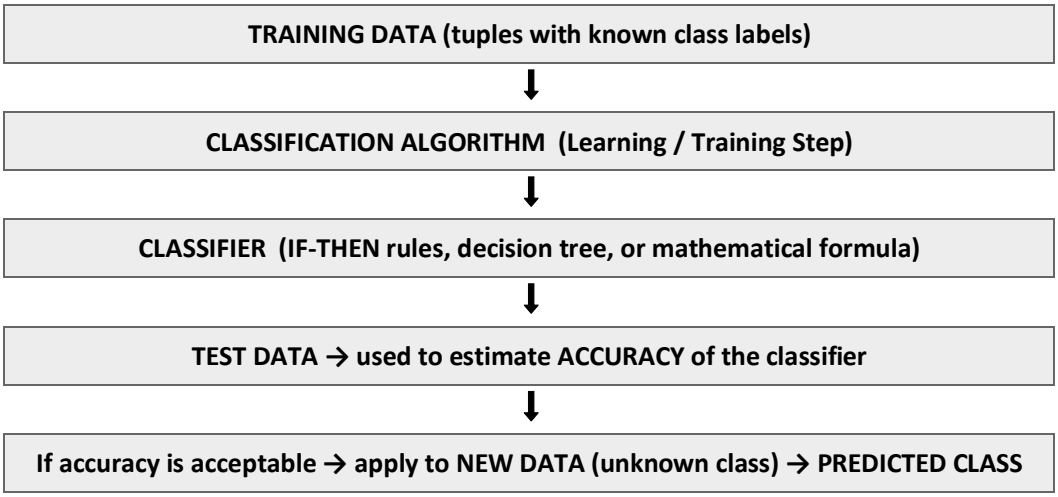
Definition

★Important Concept Classification is the process of finding a model (or function) that describes and distinguishes data classes, so as to predict the class label of objects whose class is unknown. Prediction (regression) is used to predict missing or unavailable, numeric/continuous values.

Easy Explanation

Classification = sorting into labelled boxes (e.g., which milk-fat category a packet belongs to — a fixed, predefined set of classes). Prediction/Regression = guessing a number (e.g., next month's sales). Classification predicts a CATEGORY; Prediction predicts a NUMBER.

Two-Step Process — Draw This Diagram



Step 1 — Learning/Training step: A classification algorithm analyses a training set of tuples (each already labelled with its class) and builds a classifier. Step 2 — Classification step: Test data (separate from training data) is used to estimate the classifier's accuracy; if acceptable, the classifier is used to predict class labels for new, unknown tuples.

Illustrative Example (Credit-Rating)

Name	Age	Income	Credit Rating
Suresh	<30	Low	Fair
Rakesh	<30	Low	Excellent
Mahesh	30–40	High	Fair

Name	Age	Income	Credit Rating
Sunil	>40	Medium	Excellent

Learned rule: IF Age = 30–40 AND Income = High THEN Credit Rating = Excellent. This rule is applied to a new tuple (Sumit, <30, Low) to predict its credit rating.

Supervised vs Unsupervised Learning

Basis	Supervised Learning (Classification)	Unsupervised Learning (Clustering)
Class labels	Known in advance (training data is labelled)	Not known — no predefined classes
Goal	Predict class of new/unknown tuple	Discover groups/clusters in the data
Example	Loan applicant: safe/risky	Customer segmentation

Issues Regarding Classification and Prediction

★**Important Concept** Before building and comparing classifiers, two broad sets of issues must be addressed: (A) preparing the data, and (B) choosing an evaluation basis to compare methods.

(A) Preparing the Data for Classification and Prediction

- Data Cleaning — remove or reduce noise, handle missing values in the data.
- Relevance Analysis (Feature Selection) — remove irrelevant or redundant attributes to improve efficiency and accuracy.
- Data Transformation and Reduction — normalize/generalize data so large-range attributes don't dominate; reduce dimensionality where possible.

(B) Comparing Classification and Prediction Methods

Criterion	Meaning
Accuracy	Ability of the model to correctly predict the class label of new/unseen data
Speed	Computational cost of constructing and using the model
Robustness	Ability to handle noise and missing values correctly
Scalability	Efficiency on large volumes of data
Interpretability	Level of understanding/insight the model provides to a human

RTU Exam Point "Accuracy, Speed, Robustness, Scalability, Interpretability" — remember as a mnemonic "A-S-R-S-I". This list alone is safe for a 2-4 mark sub-part of any classification question.

Quick Revision — Topic 1

- Classification = predicts a category (discrete/nominal label); Prediction/Regression = predicts a number (continuous value).
- Two-step process: Learning/Training step (build classifier from labelled training data) → Classification step (test accuracy, then apply to new data).
- Supervised (classification, labels known) vs Unsupervised (clustering, no labels).
- Issues: (A) Data prep — Cleaning, Relevance Analysis, Transformation/Reduction. (B) Comparing methods — Accuracy, Speed, Robustness, Scalability, Interpretability.

2. Decision Tree Induction

PYQ Frequency: High (2×) | RTU 2024, 2025

PYQs from This Topic

2024 (Paper-II) — 4 Marks: What are the strengths and weaknesses of decision tree induction compared to other classification methods?

2025 — 4 Marks: Explain decision tree induction with a simple example.

Teacher's bank — 2/4 Marks: What is the advantage and disadvantage of decision tree approach over other approaches of data mining? / Explain the decision tree construction.

Definition

★**Important Concept** A decision tree is a flow-chart-like tree structure used for classification (and prediction). Each internal node denotes a test on an attribute, each branch represents an outcome of the test, and each leaf (terminal) node holds a class label.

Structure — 3 Types of Nodes

Node	Meaning
Root Node	Topmost node — represents the entire dataset / the first (main) test question
Branch / Internal Node	Intermediate node — a further test/decision on another attribute
Leaf Node	Terminal node — gives the final class label (the answer)

How a Decision Tree is Built (ID3-style Algorithm)

- Start with all training tuples at the root node.
- Select the "best" splitting attribute using an attribute selection measure (Information Gain for ID3, Gain Ratio for C4.5, Gini Index for CART).
- Create a branch for each outcome/value of the chosen attribute and partition the tuples accordingly.
- Recursively repeat the same process for each partition (recursive partitioning).
- Stop when: (a) all tuples in a partition belong to the same class, OR (b) there are no remaining attributes to split on (assign majority class), OR (c) there are no tuples left for a branch.

? Attribute Selection Measure — Information Gain (most important for RTU)

$Info(D) = -\Sigma\ p_i\ \log_2(p_i)$ [Entropy of the whole dataset D]
$InfoA(D) = \Sigma\ (|D_j|\ /\ |D|) \times Info(D_j)$ [Weighted entropy after splitting on attribute A]
$Gain(A) = Info(D) - InfoA(D)$ [Information Gain of attribute A]

Logic: Lower entropy = purer/more organised data. The attribute that reduces entropy the MOST (i.e., gives the HIGHEST information gain) is chosen as the splitting attribute at that node.

? Solved Example — Entropy & Information Gain (Outlook attribute)

Dataset: 14 days of weather data used to decide whether to "Play" (Yes/No). Attribute Outlook has 3 values: Sunny (5 days: 3 Yes, 2 No), Overcast (5 days: 5 Yes, 0 No), Rainy (4 days: 2 Yes, 2 No). Overall: 10 Yes, 4 No out of 14 days.

Step 1 — Entropy of the whole dataset D

$$Info(D) = -(10/14)\log_2(10/14) - (4/14)\log_2(4/14)$$
$$= -(0.714)(-0.485) - (0.286)(-1.807)$$
$$= \mathbf{0.347 + 0.517 = 0.863\ bits}$$

Step 2 — Entropy of each Outlook value

$Info(Sunny) = -(3/5)\log_2(3/5) - (2/5)\log_2(2/5) = 0.971$
$Info(Overcast) = -(5/5)\log_2(5/5) - (0/5)\log_2(0/5) = 0$ (pure — all Yes)
$Info(Rainy) = -(2/4)\log_2(2/4) - (2/4)\log_2(2/4) = 1.000$

Step 3 — Weighted entropy after splitting on Outlook

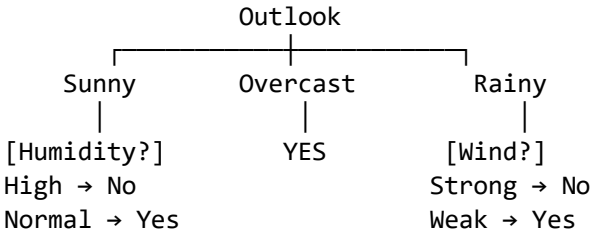
$$InfoOutlook(D) = (5/14)(0.971) + (5/14)(0) + (4/14)(1.000)$$
$$= 0.347 + 0 + 0.286 = 0.633\ bits$$

Step 4 — Information Gain

$Gain(Outlook) = Info(D) - InfoOutlook(D) = 0.863 - 0.633 = 0.230$ bits

Similarly, Gain is computed for the other candidate attributes (Temperature, Humidity, Wind). Whichever attribute gives the HIGHEST gain is chosen as the root — here Outlook has a strong gain (Overcast is already "pure"), so Outlook is a strong candidate for the root node.

Resulting Simple Decision Tree — Draw This in Exam



Alternate Diagram Style (from class notes — equally valid)

```
Root Node → AGE
  Age < 30      → Income < 30k → Bad;  Income ≥ 30k → Good
  Age 30–50     → Income node → Bad / Average
  Age 50–60     → Income node → Good / Bad
  Age > 60      → Income node → Average / Good
```

Rule extracted: IF Age > 30 AND Income ≥ 30k THEN Group = B (Good). Decision trees are essentially a visual, human-readable representation of a set of classification rules.

Other Splitting Measures (bonus — mention for extra marks)

Gini Index (used in CART): $Gini(D) = 1 - \Sigma\ (p_i)^2$ — lower Gini = purer node

Overfitting & Tree Pruning

- Decision trees can overfit noisy/too-detailed training data, creating an overly complex tree that generalises poorly.
- Pre-pruning — halt tree construction early (before it is fully grown) using a stopping condition.
- Post-pruning — remove branches from a fully-grown tree based on statistical/cost-complexity measures (more common and usually more accurate).

Strengths vs Weaknesses (directly answers a 2024 PYQ)

Strengths	Weaknesses
Easy to understand, interpret and visualize (white-box model)	Prone to overfitting, especially with deep trees on noisy data
Requires little data preparation — no normalization/scaling needed	Can be unstable — a small change in data may produce a very different tree
Handles both numerical and categorical attributes	Biased towards attributes with many distinct values (Information Gain bias — fixed by Gain Ratio in C4.5)
Non-parametric — no assumption about data distribution	Large trees can be hard to interpret and may need pruning
Rules can be directly extracted from root-to-leaf paths	Not always the most accurate compared to ensembles (Random Forests, Boosting)

RTU Exam Point For scalability to large databases, mention SLIQ and SPRINT (decision-tree algorithms specifically designed for disk-resident, very large training sets) — a good bonus line in a 10-mark answer.

Practice Question

Q: A dataset of 10 tuples has 6 "Yes" and 4 "No". Find Info(D) (entropy of the dataset).

A: $Info(D) = -(6/10)log_2(6/10) - (4/10)log_2(4/10) = -(0.6)(-0.737) - (0.4)(-1.322) = 0.442 + 0.529 = 0.971$ bits.

Quick Revision — Topic 2

- 3 node types: Root (main test), Branch/Internal (further test), Leaf (final class label).
- Algorithm: pick best splitting attribute (highest Information Gain) → branch on its values → recurse → stop when pure/no attributes/no tuples left.
- $Info(D) = -\Sigma p_i log_2(p_i)$; $InfoA(D) = \Sigma(|D_j|/|D|) \times Info(D_j)$; $Gain(A) = Info(D) - InfoA(D)$. Highest Gain wins.
- Gini Index (CART) = $1 - \Sigma(p_i)^2$ is the alternative measure.
- Overfitting fixed by Pre-pruning (stop early) or Post-pruning (trim a fully grown tree).
- Strengths: interpretable, no scaling needed, handles mixed data. Weaknesses: overfits, unstable, biased toward multi-valued attributes.

3. Bayes Classification Methods (Naïve Bayesian Classification)

PYQ Frequency: High | RTU 2025

PYQs from This Topic

2025 — 4 Marks: Write a note on Bayes classification methods.

Teacher's bank — 4 Marks: Explain the Naive Bayes classification.

Definition

Important Concept Bayesian classifiers are statistical classifiers that predict class-membership probabilities — the probability that a given tuple belongs to a particular class. They are based on Bayes' Theorem. The Naïve Bayesian classifier further assumes class-conditional independence — the effect of one attribute's value on a given class is independent of the values of the other attributes (the "naïve" assumption) — which greatly simplifies computation.

Bayes' Theorem

$$P(H|X) = [\ P(X|H) \times P(H)\ ] / P(X)$$

Term	Meaning
$P(H\|X)$	Posterior probability — probability that hypothesis H holds, given the observed data tuple X
$P(H)$	Prior probability — probability of H being true, before seeing X
$P(X\|H)$	Likelihood — probability of observing X, given that H is true
$P(X)$	Evidence — prior probability of X (same for all classes, can be treated as a constant)

Naïve Bayes Classifier — Steps

- Let D be a training set of tuples, each with class labels; let $C_1, C_2, \ldots, C_n$ be the possible classes.
- Given a tuple X, the classifier predicts X belongs to the class C_i having the highest posterior probability $P(C_i|X)$, i.e., maximise $P(X|C_i) \times P(C_i)$ ($P(X)$ is the same for every class, so it can be ignored while comparing).
- Class prior probability: $P(C_i) = |C_i,D| / |D|$ (fraction of training tuples in class C_i).
- Naïve (class-conditional independence) assumption: $P(X|C_i) = P(x_1|C_i) \times P(x_2|C_i) \times \ldots \times P(x_n|C_i)$ — the product of the probabilities of each individual attribute value given the class.
- For a categorical attribute: $P(x_n|C_i)$ = (no. of tuples of class C_i having value x_n) / $|C_i,D|$.
- For a continuous attribute: assume a Gaussian (normal) distribution and use the class mean & standard deviation in the density function.
- Predicted class = the C_i for which $P(X|C_i) \times P(C_i)$ is maximum.

Solved Example — Naïve Bayes on the Play/Outlook dataset

Same 14-day weather dataset as Topic 2. Question: If the weather (Outlook) is Sunny, should the players play or not?

Step 1 — Frequency Table for Outlook

Outlook	Play = Yes	Play = No
Overcast	5	0
Rainy	2	2
Sunny	3	2

Outlook	Play = Yes	Play = No
Total	10	4

Step 2 — Likelihood Table

Outlook	P(Outlook | Yes)	P(Outlook | No)
Sunny	3/10 = 0.30	2/4 = 0.50
P(class)	P(Yes) = 10/14 = 0.71	P(No) = 4/14 = 0.29
P(Sunny) (overall)	5/14 = 0.35	5/14 = 0.35

Step 3 — Apply Bayes' Theorem

`P(Yes|Sunny) = [P(Sunny|Yes) × P(Yes)] / P(Sunny) = (0.30 × 0.71) / 0.35 = 0.213/0.35 = 0.60`
`P(No|Sunny) = [P(Sunny|No) × P(No)] / P(Sunny) = (0.50 × 0.29) / 0.35 = 0.145/0.35 = 0.41`

Since P(Yes|Sunny) = 0.60 > P(No|Sunny) = 0.41 → Prediction: YES, the players will play on a sunny day.

? Practice Question

Q: Using the same table, predict for Outlook = Overcast (should they play?).

A: P(Overcast|Yes)=5/10=0.5, P(Overcast|No)=0/4=0, P(Overcast)=5/14=0.357. So P(Yes|Overcast) = (0.5×0.71)/0.357 = 1.0 and P(No|Overcast) = 0. Prediction: YES (certain) — Overcast never resulted in "No" in the training data.

? **Common Mistake** If a likelihood value is exactly 0 (as above for P(Overcast|No)), it can wrongly force the whole product to 0 even when other evidence suggests otherwise. RTU examiners like to see this flagged — mention the Laplacian (additive) correction/smoothing, which adds a small count to every attribute-value/class combination so no probability is ever exactly zero.

Advantages & Disadvantages

Advantages	Disadvantages
Simple, fast, and easy to implement	Assumes class-conditional independence of attributes (rarely fully true in real data)
Works well even with large / high-dimensional datasets	Zero-frequency problem for unseen attribute values (fixed via Laplacian smoothing)
Performs surprisingly well in practice (e.g., text classification) despite the naïve assumption	Accuracy can drop when attributes are strongly correlated

Applications: Spam email filtering, sentiment analysis, text/document classification, medical diagnosis.

? **Quick Revision — Topic 3**

- Bayes' theorem: P(H|X) = P(X|H)P(H) / P(X).
- Naïve Bayes assumes class-conditional independence: $P(X|C_i) = \Pi P(x?|C_i)$.
- Predict the class C_i that maximises $P(X|C_i) \times P(C_i)$ (P(X) can be dropped — same for all classes).
- Worked example: P(Yes|Sunny)=0.60 > P(No|Sunny)=0.41 → Play = Yes.
- Zero probability issue → fixed by Laplacian correction.
- Advantages: simple, fast, scales well. Disadvantage: independence assumption rarely holds exactly.

4. Bayesian Belief Networks (Advanced Classification Method)

PYQ Frequency: Very High (2×, both 10 Marks) | RTU 2024 (both papers) — HIGHEST-YIELD topic of Unit 2

PYQs from This Topic

2024 (Paper-I) — 10 Marks: What is Bayesian belief network? Explain in detail.

2024 (Paper-II) — 10 Marks: Explain Bayesian belief networks with structure, working mechanism, and advantages in advanced classification methods.

Teacher's bank — 4/10 Marks: Briefly outline the major idea of Bayesian Belief network classification.

Frequently Asked This exact topic has appeared as a 10-mark question in BOTH 2024 papers, worded differently. Treat it as a guaranteed Part-C question — memorise the DAG diagram, the CPT concept, and the full worked numerical below.

Definition

★**Important Concept** A Bayesian Belief Network (BBN) — also called a Bayesian Network, Belief Network, or Probabilistic Network — is a probabilistic graphical model (PGM) that represents conditional dependencies between a set of random variables through a Directed Acyclic Graph (DAG). Unlike the Naïve Bayes classifier (which assumes ALL attributes are class-conditionally independent), a BBN allows the specification of class-conditional independencies between SUBSETS of variables — making it more flexible and realistic.

Two Main Components

(a) Directed Acyclic Graph (DAG) — the Network Topology

- Each node represents a random variable / hypothesis (can be a real attribute or a hidden/latent variable).
- Each directed edge (arrow) represents a probabilistic dependency of the child node on its parent node(s).
- No directed cycles are allowed — hence "acyclic" (you can never follow the arrows back to where you started).
- A node with no parents carries only a prior probability; a node WITH parents needs a Conditional Probability Table.

Simple Illustrative DAG (from class notes)

`Rain → Dog Barks → Cat Hides`

(When it rains, the dog is likely to bark; when the dog barks, the cat is likely to hide. All arrows point forward — no loop — so it is acyclic.)

(b) Conditional Probability Table (CPT)

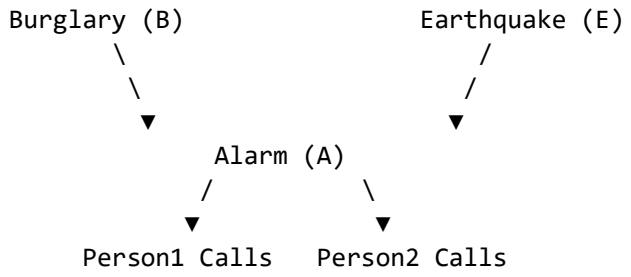
- For every node, the CPT gives P(node | its parents) for every combination of parent values.
- If a node has no parents, its CPT is simply its prior probability P(node).

Joint probability formula: $P(x_1, x_2, \ldots, x_n) = \prod P(x_i \mid \text{Parents}(x_i)),$ **for** $i = 1$ **to** n

This lets us compute the probability of ANY combination of events in the network just by multiplying the relevant CPT entries — this is the key working mechanism examiners want to see explained.

? Solved Example — Burglary–Earthquake–Alarm Network (classic RTU example)

Network Structure — Draw This Diagram



Here Burglary (B) and Earthquake (E) are independent root nodes (only priors). Alarm (A) depends on BOTH B and E. Person1Calls (P1) and Person2Calls (P2) each depend only on Alarm.

Given Prior Probabilities

Event	P(True)	P(False)
Burglary (B)	0.001	0.999
Earthquake (E)	0.002	0.998

CPT of Alarm (given parents B, E)

Burglary	Earthquake	P(Alarm=T)	P(Alarm=F)
T	T	0.95	0.05
T	F	0.94	0.06
F	T	0.29	0.71
F	F	0.001	0.999

CPT of Person1Calls & Person2Calls (given Alarm)

Alarm	P(P1=T)	P(P1=F)	P(P2=T)	P(P2=F)
T	0.90	0.10	0.70	0.30
F	0.05	0.95	0.01	0.99

Query: Find P(P1 = T, P2 = T, Alarm = T, Burglary = F, Earthquake = F)

$$P(P1, P2, A, \neg B, \neg E) = P(P1|A) \times P(P2|A) \times P(A|\neg B,\neg E) \times P(\neg B) \times P(\neg E)$$
$$= 0.90 \times 0.70 \times 0.001 \times 0.999 \times 0.998$$
$$= \mathbf{0.00062} \quad (\approx \mathbf{6.2 \times 10^{-4}})$$

Logic: The joint probability of the whole scenario is just the product of each node's CPT entry (given its own parents) — this is exactly the chain-rule formula shown above, applied node by node.

Advantages of Bayesian Belief Networks

- Combines prior domain/expert knowledge with observed data — more realistic than Naïve Bayes since it does NOT assume full attribute independence.
- Gives a compact, visual representation of the joint probability distribution of many variables.

- Handles incomplete datasets reasonably well.
- Models real causal relationships, giving insight into a problem domain.
- Combines naturally with standard statistical/Bayesian data-analysis methods.

Naïve Bayes vs Bayesian Belief Network

Basis	Naïve Bayes	Bayesian Belief Network
Independence assumption	ALL attributes assumed class-conditionally independent	Only some variables independent, as defined by the graph — allows real dependencies
Representation	Simple probability table, no graph	DAG + a Conditional Probability Table per node
Realism	Less realistic	More realistic — models actual causal structure
Computation	Simple and fast	More complex, computationally heavier

Applications: Medical diagnosis, risk/security analysis (as in the burglar-alarm example), gene expression/bioinformatics, decision-support systems.

⍰ Quick Revision — Topic 4 (Very High Priority)

- BBN = probabilistic graphical model = DAG (nodes = random variables, edges = dependency) + CPT (P(node|parents) for each node).
- No directed cycles allowed. Root nodes (no parents) have only priors.
- Joint probability formula: $P(x_1,\ldots,x_n) = \Pi\, P(x_i|\text{Parents}(x_i))$.
- Classic example: Burglary & Earthquake → Alarm → Person1Calls, Person2Calls. Query answer = 0.00062 (multiply the relevant CPT entries).
- BBN is more realistic than Naïve Bayes since it allows dependencies between variables instead of assuming full independence.
- Always draw the DAG + at least one CPT + the joint-probability formula for full marks in a 10-mark answer.

5. Classification by Backpropagation

⍰ PYQ Frequency: Medium-High | RTU 2024 — numerical-heavy, strong 10m upgrade candidate

⍰ PYQs from This Topic

2024 (Paper-II) — 2 Marks: Mention any one advantage and one limitation of classification using back-propagation.

Teacher's bank — 4/10 Marks: Write short notes on Back Propagation algorithm with its example.

Definition

★Important Concept Backpropagation ("backward propagation of errors") is a neural-network learning algorithm. A Neural Network is a set of connected input/output units in which each connection has an associated weight. During the learning phase, the network learns by adjusting the weights so it can correctly predict the class label of the input tuples. This is also called "connectionist learning" because of the connections between units.

Network Structure — Multilayer Feed-Forward Neural Network

Layer	Role
Input Layer	One node per input attribute; simply feeds the input values forward
Hidden Layer(s)	Each unit computes a weighted sum of inputs + bias, then applies an activation function (usually Sigmoid)
Output Layer	Produces the final class prediction(s)

```
x1, x2  →(weights w1..w4, bias b1)→  h1, h2 (hidden)  →(weights w5..w8, bias b2)→  o1, o2 (output)
```

Algorithm — Steps

- Initialise all weights and biases to small random values.
- Forward Propagation — for every hidden/output unit compute the net input as a weighted sum plus bias, then apply the activation (sigmoid) function:

$net = \Sigma(w_i \times x_i) + b$ $\quad$ $Output = sigmoid(net) = 1 / (1 + e^{-net})$

- Compute the Error — compare the network's actual output with the target (desired) output:

$E_{total} = \Sigma\ (1/2)(target - output)^2$

- Backpropagate the Error — propagate the error backward from the output layer towards the hidden layer, using gradient descent to update each weight:

$w_{new} = w_{old} - \eta \times (\partial E_{total} / \partial w)$ $\quad$ where η = learning rate

- Repeat the forward + backward pass (an "epoch") over the training data until the total error is acceptably small (the network converges).

? Solved Numerical Example — Forward Pass & Error

Given: inputs x1=0.05, x2=0.10; input→hidden weights w1=0.15, w2=0.20, w3=0.25, w4=0.30; hidden→output weights w5=0.40, w6=0.45, w7=0.50, w8=0.55; biases b1=0.35 (hidden layer), b2=0.60 (output layer); targets: target_o1=0.01, target_o2=0.99.

Step 1 — Hidden layer net input & output

```
h1(in) = w1·x1 + w2·x2 + b1 = 0.15×0.05 + 0.20×0.10 + 0.35 = 0.3775
h1(out) = 1/(1+e^-0.3775) = 0.5933
h2(in) = w3·x1 + w4·x2 + b1 = 0.25×0.05 + 0.30×0.10 + 0.35 = 0.3925
h2(out) = 1/(1+e^-0.3925) = 0.5969
```

Step 2 — Output layer net input & output

```
o1(in) = w5·h1(out) + w6·h2(out) + b2 = 0.40×0.5933 + 0.45×0.5969 + 0.60 = 1.1059
o1(out) = 1/(1+e^-1.1059) = 0.7514
o2(in) = w7·h1(out) + w8·h2(out) + b2 = 0.50×0.5933 + 0.55×0.5969 + 0.60 = 1.2249
o2(out) = 1/(1+e^-1.2249) = 0.7729
```

Step 3 — Total Error

```
E_o1 = (1/2)(0.01 - 0.7514)² = 0.2748
E_o2 = (1/2)(0.99 - 0.7729)² = 0.0235
```

E_total = E_o1 + E_o2 = 0.2748 + 0.0235 = 0.2983

Since the outputs (0.7514, 0.7729) do not match the targets (0.01, 0.99), a large error exists. Backpropagation now works BACKWARD from the output layer, using the weight-update formula w_new = w_old − η(∂E_total/∂w) to adjust w5, w6, w7, w8 first (output→hidden), and then w1–w4 (hidden→input), via the chain rule. This forward+backward cycle is repeated over many epochs until the error becomes acceptably small.

Advantages & Limitations (directly answers the 2024 PYQ)

Advantages	Limitations
Can model complex, non-linear relationships between inputs and outputs	Long training time, especially for large networks/datasets
High tolerance to noisy data	Requires many parameters best determined empirically (topology, number of hidden units, learning rate)
Can classify patterns it has not been explicitly trained on (good generalisation)	Poor interpretability — a "black box"; hard to interpret the meaning of learned weights
Well suited to continuous-valued inputs and outputs	Needs a large amount of training data for good accuracy
Algorithms are inherently parallelizable, which has spurred hardware-based (GPU) speed-up	Can get stuck in local minima during gradient descent

Quick Revision — Topic 5

- Backpropagation = neural network learning algorithm; adjusts connection weights to minimise output error ("connectionist learning").
- 3 layers: Input → Hidden (weighted sum + bias + sigmoid) → Output.
- Forward pass: net = Σ(w·x)+b, output = sigmoid(net). Error: E_total = Σ½(target−output)².
- Backward pass: w_new = w_old − η(∂E_total/∂w); repeated over many epochs until convergence.
- Worked example gives E_total ≈ 0.2983 after just one forward pass — shows why many epochs are needed.
- Advantage: handles non-linear/noisy data well. Limitation: slow training, black-box (poor interpretability).

6. Support Vector Machines (SVM)

PYQ Frequency: Medium-High | RTU 2025

PYQs from This Topic

2025 — 4 Marks: Describe Support Vector Machines (SVMs).

Teacher's bank — 4/10 Marks: Explain support vector machine with its types. Also explain its application.

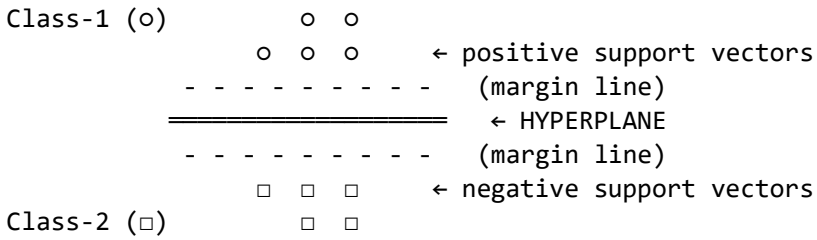
Definition

> ⋆**Important Concept** SVM is a supervised learning algorithm used for both classification and regression problems (mainly classification). The goal of the SVM algorithm is to create the best decision boundary — called a hyperplane — that can segregate an n-dimensional space into classes, so new data points can be put into the correct category in the future.

Key Terms

Term	Meaning
Hyperplane	The best decision boundary/line separating the classes. In n-dimensional space it has (n−1) dimensions.
Support Vectors	The extreme data points closest to the hyperplane; they directly determine its position — hence the algorithm's name.
Margin	The distance between the hyperplane and the nearest points (support vectors) of each class. SVM chooses the hyperplane that MAXIMISES this margin for the best generalisation.

Diagram — Draw This in Exam



The hyperplane sits exactly in the middle of the margin, maximising the gap between the two nearest points of each class — this maximum-margin property is what makes SVM a robust classifier.

Types of SVM

- **(a) Linear SVM —** used when the data is linearly separable, i.e., the dataset can be classified into two classes using a single straight line/hyperplane. The classifier used is called a Linear SVM Classifier.
- **(b) Non-linear SVM —** used when the data is NOT linearly separable, i.e., a straight line cannot classify the dataset. The classifier used is called a Non-linear SVM. It uses the "Kernel Trick" to map data into a higher-dimensional space where it becomes linearly separable (e.g., Polynomial kernel, Gaussian/RBF kernel).

Advantages & Disadvantages

Advantages	Disadvantages
Effective and accurate in high-dimensional spaces	Training can be slow / less suitable for very large datasets
Memory-efficient — uses only a subset of training points (support vectors)	Performance drops with overlapping classes / heavy noise
Versatile — different kernel functions handle different data shapes	Choosing the right kernel and parameters (C, gamma) can be tricky

Applications

- Face detection / face observation
- Image classification
- Text and hypertext categorisation (spam filtering, document classification)
- Bioinformatics — protein fold/remote homology detection, DNA/medical-science research
- Handwriting recognition
- Generalized Predictive Control (GPC) in control systems

Quick Revision — Topic 6

• SVM = supervised algorithm for classification (& regression) — finds the maximum-margin hyperplane separating classes.

• Support vectors = extreme points closest to the hyperplane; they define its position.

• Margin = distance from hyperplane to nearest support vectors — SVM maximises this.

• Linear SVM → linearly separable data; Non-linear SVM → uses kernel trick (polynomial/RBF) for non-separable data.

• Applications: face detection, text categorisation, bioinformatics, handwriting recognition.

7. Lazy Learners (vs Eager Learners)

PYQ Frequency: Medium | RTU 2024

PYQs from This Topic

2024 (Paper-II) — 2 Marks: How do lazy learners differ from eager learners?

Definition

★Important Concept A lazy learner (instance-based learner) simply stores the training tuples (or does only minimal processing) and waits until it is given a test/query tuple, before doing any real computation to classify it — typically by comparing it to the stored training tuples using a similarity/distance measure. An eager learner, given a set of training tuples, constructs a generalised classification model BEFORE receiving any new tuples to classify.

Example of a Lazy Learner — k-Nearest Neighbour (k-NN)

k-NN classifies an unknown tuple by comparing it (using a distance measure such as Euclidean distance) with the k "closest" training tuples, and assigns it the majority class among those k neighbours. Case-Based Reasoning (CBR) is another lazy learner — it reasons from stored, similar past cases/experiences rather than pure geometric distance.

Lazy vs Eager Learners — Comparison Table

Basis	Lazy Learners	Eager Learners
Model building	No explicit model built in advance — computation is deferred to query time	Builds an explicit, generalised model before any test data arrives
Training time	Very fast / negligible (just stores the data)	Slower — must build the complete model upfront

Basis	Lazy Learners	Eager Learners
Classification (query) time	Slow — must compute similarity to (potentially) all stored tuples for every query	Fast — simply applies the already-built model
Examples	k-Nearest Neighbour (k-NN), Case-Based Reasoning (CBR)	Decision Tree, Naïve Bayes, Bayesian Belief Network, Backpropagation, SVM
Storage requirement	High — must retain the entire training set	Lower — only the final model needs to be stored
Effective hypothesis space	Can implicitly use a rich, query-specific (local) hypothesis for each new tuple	Uses one single, global generalised model for the whole instance space

RTU Exam Point One-line answer for 2 marks: "Lazy learners defer computation to classification time and are slow at prediction but fast to train (e.g., k-NN); eager learners build a model upfront, so they are slower to train but fast at prediction (e.g., Decision Tree, Naïve Bayes)."

Applications: k-NN — recommendation systems, handwriting/pattern recognition. CBR — legal reasoning, medical diagnosis, customer-support systems that retrieve similar past cases.

Quick Revision — Topic 7

- Lazy learner = stores training data, delays computation until a query/test tuple arrives (e.g., k-NN, CBR).
- Eager learner = builds a generalised model BEFORE test data arrives (e.g., Decision Tree, Naïve Bayes, BBN, Backpropagation, SVM).
- Lazy: fast training, slow prediction, high storage. Eager: slow training, fast prediction, low storage.
- k-NN classifies by majority vote among the k nearest (by distance) training tuples.

PART 2 — Complete Unit 2 PYQ Bank

Long Answer Questions (10 Marks)

- What is Bayesian belief network? Explain in detail. (RTU 2024 – Paper I)
- Explain Bayesian belief networks with structure, working mechanism, and advantages in advanced classification methods. (RTU 2024 – Paper II)

Short / Medium Answer Questions (4 Marks)

- What are the strengths and weaknesses of decision tree induction compared to other classification methods? (RTU 2024 – Paper II)
- Explain decision tree induction with a simple example. (RTU 2025)
- Write a note on Bayes classification methods. (RTU 2025)
- Describe Support Vector Machines (SVMs). (RTU 2025)

Very Short Answer Questions (2 Marks)

- Mention any one advantage and one limitation of classification using back-propagation. (RTU 2024 – Paper II)
- How do lazy learners differ from eager learners? (RTU 2024 – Paper II)
- What is classification in predictive modeling? (RTU 2025)

Repeated / Similar Questions (grouped by concept)

Concept	Appearances
Bayesian Belief Networks	Asked TWICE at 10 marks in 2024 (both papers) — different wording, same core answer
Decision Tree Induction	Asked in both 2024 (strengths/weaknesses) and 2025 (explain with example) at 4 marks

Most Important Questions (Highest confidence for next exam)

- Explain Bayesian Belief Networks — structure (DAG), CPT, working mechanism, and a solved numerical example
- Decision Tree Induction — construction steps + Information Gain/Entropy numerical + strengths/weaknesses
- Bayes Classification Methods — Bayes' theorem + Naïve Bayes worked example
- Support Vector Machines — hyperplane, margin, support vectors, types, applications
- Classification by Backpropagation — algorithm steps + advantage/limitation (numerical if 4m/10m)
- Lazy vs Eager Learners — comparison table

Expected Questions (not yet asked at high marks, high probability)

- Write and explain the concept of classification and prediction along with various issues incorporated with them (10m — appeared in teacher's older question bank, matches RTU's general-approach pattern)
- Explain the general approach to classification (two-step process) with a diagram (4m/10m — currently only a 2m PYQ, due for an upgrade)

- Explain Naïve Bayes classification with a fully solved numerical (10m — currently only a 4m "note")
- Compare Naïve Bayes and Bayesian Belief Networks
- Explain the Backpropagation algorithm with a complete forward-and-backward-pass numerical (10m — currently only 2m, numerical-heavy topics are a classic RTU upgrade pattern)
- Compare Linear and Non-linear SVM with examples

PART 3 — Model Answers

Grouped by concept — matched to the marks weight so you don't over- or under-write in the exam.

Q. What is classification in predictive modeling? (2m)

Classification is the process of finding a model that describes and distinguishes data classes, so as to predict the class label of objects whose class is unknown, based on a training dataset of tuples with known class labels. It is one of the two major tasks of predictive modeling (the other being regression/prediction, which predicts continuous values).

Q. How do lazy learners differ from eager learners? (2m)

Lazy learners (e.g., k-Nearest Neighbour) simply store the training tuples and defer all computation until a test tuple is given, then classify it based on similarity to stored data — fast to train but slow to classify. Eager learners (e.g., Decision Tree, Naïve Bayes, SVM, Backpropagation) build a generalised model BEFORE any test data arrives — slower to train but fast to classify.

Q. Mention one advantage and one limitation of classification using backpropagation. (2m)

Advantage: Backpropagation (neural network) classifiers can model complex, non-linear relationships in the data and have high tolerance to noisy inputs. Limitation: They require a long training time and act as a "black box" with poor interpretability, since it is hard to explain the meaning of the learned weights.

Q. What are the strengths and weaknesses of decision tree induction compared to other classification methods? / Explain decision tree induction with a simple example. (4m)

A decision tree is a flow-chart-like tree structure where internal nodes test an attribute, branches represent outcomes, and leaf nodes give the class label. It is built (ID3-style) by recursively selecting the attribute with the highest Information Gain, Gain(A) = Info(D) − InfoA(D), to split the data at each node, continuing until nodes become pure or run out of attributes.

Example: On a 14-day weather dataset (Outlook: Sunny/Overcast/Rainy) with 10 "Yes" and 4 "No", Info(D) = 0.863 bits. Splitting on Outlook gives InfoOutlook(D) = 0.633 bits, so Gain(Outlook) = 0.230 bits — since Overcast is already pure (5 Yes, 0 No), Outlook is a strong candidate for the root node.

Strengths: easy to interpret and visualise, needs little data preparation, handles both numeric and categorical attributes. Weaknesses: prone to overfitting (fixed by pre/post pruning), can be unstable to small data changes, and is biased toward attributes with many distinct values.

Q. Write a note on Bayes classification methods. (4m)

Bayesian classifiers are statistical classifiers that predict class-membership probabilities using Bayes' theorem: $P(H|X) = P(X|H)P(H)/P(X)$. The Naïve Bayesian classifier additionally assumes class-conditional independence of attributes, so $P(X|C_i) = \Pi\ P(x_k|C_i)$, which greatly simplifies computation — the class C_i maximising $P(X|C_i)\times P(C_i)$ is predicted.

Example: For the weather dataset, given Outlook = Sunny, $P(Yes|Sunny) = (0.30 \times 0.71)/0.35 = 0.60$ and $P(No|Sunny) = (0.50 \times 0.29)/0.35 = 0.41$, so the classifier predicts "Yes, play" since $0.60 > 0.41$.

Advantages: simple, fast, works well on large/high-dimensional data (e.g., spam filtering). Disadvantage: the independence assumption is rarely fully true, and zero-probability values require Laplacian correction.

Q. Describe Support Vector Machines (SVMs). / Explain SVM with its types and applications. (4m)

SVM is a supervised learning algorithm used for classification and regression. It finds the best decision boundary — the hyperplane — that maximises the margin (distance) between itself and the nearest data points of each class, called support vectors, so new points can be reliably classified.

Types: Linear SVM is used when data is linearly separable by a single straight line/hyperplane. Non-linear SVM is used when data cannot be separated linearly; it uses the kernel trick (e.g., polynomial or RBF kernel) to map data into a higher-dimensional space where it becomes separable.

Applications: face detection, image classification, text/hypertext categorisation, bioinformatics (protein classification), and handwriting recognition.

Q. What is Bayesian belief network? Explain in detail (structure, working mechanism, advantages). / Briefly outline the major idea of Bayesian belief network classification. (10m)

A Bayesian Belief Network (BBN) is a probabilistic graphical model that represents conditional dependencies among random variables through a Directed Acyclic Graph (DAG), where nodes are variables and directed edges show probabilistic dependency. Unlike Naïve Bayes, which assumes full attribute independence, a BBN allows dependencies between subsets of variables via a Conditional Probability Table (CPT) attached to every node — CPT(node) = P(node | its parents).

Working mechanism: the joint probability of any assignment of values to all variables is computed as the product of each node's CPT entry given its own parents: $P(x_1,\ldots,x_n) = \Pi\, P(x_i|\text{Parents}(x_i))$. For example, in the classic Burglary–Earthquake–Alarm–Calls network, given P(B=T)=0.001, P(E=T)=0.002, and the CPTs of Alarm, Person1Calls and Person2Calls, the probability P(P1=T, P2=T, A=T, B=F, E=F) = $P(P1|A)\times P(P2|A)\times P(A|\neg B,\neg E)\times P(\neg B)\times P(\neg E) = 0.90\times 0.70\times 0.001\times 0.999\times 0.998 \approx 0.00062$.

Advantages: combines expert/prior knowledge with observed data; gives a compact visual representation of the full joint distribution; handles incomplete data; models real causal relationships (unlike the unrealistic full-independence assumption of Naïve Bayes). Applications: medical diagnosis, security/risk analysis, bioinformatics, decision-support systems.

(Draw the DAG structure and at least one CPT table, then show the joint-probability computation, for full marks.)

PART 4 — Final Revision Material

? 1-Day Revision Sheet

- Classification = predicts a category; Prediction/Regression = predicts a number. Two-step process: Learning step (build classifier) → Classification step (test accuracy, then apply).
- Issues: Data prep (Cleaning, Relevance Analysis, Transformation/Reduction); Compare methods (Accuracy, Speed, Robustness, Scalability, Interpretability).
- Decision Tree: Root/Branch/Leaf nodes. Best split = highest Information Gain = Info(D) − InfoA(D), where Info(D) = $-\Sigma p_i \log_2(p_i)$. Gini Index (CART) = $1-\Sigma(p_i)^2$. Overfitting fixed by pre/post pruning.
- Naïve Bayes: $P(H|X) = P(X|H)P(H)/P(X)$. Class-conditional independence: $P(X|C_i) = \Pi P(x?|C_i)$. Predict class with max $P(X|C_i)\times P(C_i)$. Zero-probability → Laplacian correction.
- Bayesian Belief Network: DAG + CPT. Joint probability = Π P(node|parents). More realistic than Naïve Bayes — allows partial dependence, not full independence. Practice the Burglary-Alarm numerical (answer ≈ 0.00062).
- Backpropagation: Neural network — Input → Hidden (weighted sum+bias+sigmoid) → Output. E_total = Σ½(target−output)². Weight update: w_new = w_old − η(∂E_total/∂w).
- SVM: Maximum-margin hyperplane; support vectors = closest points. Linear SVM (linearly separable) vs Non-linear SVM (kernel trick for non-separable data).
- Lazy Learners (k-NN, CBR): defer computation to query time — fast train, slow predict. Eager Learners (Decision Tree, Naïve Bayes, BBN, Backprop, SVM): build model upfront — slow train, fast predict.

? One-Night-Before Ultra-Short Notes

Term	One-liner
Classification	Predicts a CATEGORY (discrete label)
Prediction/Regression	Predicts a NUMBER (continuous value)
Decision Tree	Root → Branch → Leaf; split by highest Information Gain
Entropy Info(D)	$-\Sigma p_i \log_2(p_i)$ — measures impurity/disorder
Information Gain	Info(D) − InfoA(D) — pick attribute with the highest value
Gini Index	$1 - \Sigma(p_i)^2$ — used by CART
Naïve Bayes	$P(H\|X)=P(X\|H)P(H)/P(X)$; assumes attribute independence
Bayesian Belief Network	DAG + CPT; joint prob. = Π P(node|parents); allows real dependencies
Backpropagation	Neural network; adjusts weights backward to reduce error
Sigmoid function	$1/(1+e^{-net})$ — squashes output between 0 and 1
SVM Hyperplane	Best decision boundary; maximises margin to support vectors
Support Vectors	Extreme points closest to the hyperplane
Linear vs Non-linear SVM	Straight-line separable data vs kernel-trick for non-separable data
Lazy Learner	Stores data, computes only at query time (e.g., k-NN)
Eager Learner	Builds model before query time (e.g., Decision Tree, SVM)

? Top 15 Most Important Unit 2 Questions (Ranked)

- **1.** Explain Bayesian Belief Networks — structure (DAG+CPT), working mechanism, and a solved numerical example. (10m — asked twice in 2024)

- **2.** Explain decision tree induction with a simple example / with strengths and weaknesses. (4m — asked in both 2024 and 2025)
- **3.** Write a note on Bayes classification methods (Naïve Bayes, with Bayes' theorem and a solved example). (4m)
- **4.** Describe Support Vector Machines — hyperplane, support vectors, margin, types, applications. (4m)
- **5.** Mention advantages and limitations of classification using backpropagation. (2m — strong candidate to upgrade to 4m/10m with the numerical)
- **6.** How do lazy learners differ from eager learners? (2m)
- **7.** What is classification in predictive modeling? (2m)
- **8.** Explain the general approach to classification (two-step process) with a diagram. (Expected 4m/10m)
- **9.** Discuss the issues regarding classification and prediction (data preparation + comparing methods). (Expected 4m)
- **10.** Explain the Information Gain / Entropy calculation with a solved numerical. (Expected 4m/10m numerical)
- **11.** Compare Naïve Bayes and Bayesian Belief Networks. (Expected 4m)
- **12.** Explain the Backpropagation algorithm with a full forward-and-backward-pass numerical. (Expected 10m)
- **13.** Compare Linear and Non-linear SVM with examples. (Expected 4m)
- **14.** Explain the advantages and disadvantages of decision trees compared to other classifiers. (4m — asked 2024)
- **15.** Explain the working of the Naïve Bayes classifier step by step. (Expected 4m/10m)

✅**Final Word** Bayesian Belief Networks is the single highest-yield topic in Unit 2 — a guaranteed 10-marker two years running. Follow it up with Decision Tree Induction and Naïve Bayes (both numerical-friendly, both repeated) — together these three topics alone can secure 60-70% of Unit 2 marks. Don't skip Backpropagation, SVM, and Lazy Learners though — they are classic short-answer (Part A/B) certainties. All the best for your RTU exam!